AI Engineer 자기소개서

황화인

목차

1. ALL_FOR_ONE ..... p. 2
좋은 기법이 내 문제에서도 좋은지 확인했습니다

2. MEMORIA LABYRINTH ..... p. 3
응답의 문제를 기억이 만들어지는 단계까지 내려가 해결했습니다

3. 베스핀글로벌 Copilot 평가·성능 개선 ..... p. 4
평가 기준과 개발 Agent의 실행 경로를 함께 관리했습니다

4. Q&A 게시판 ..... p. 6
요구사항이 바뀌어도 코딩 Agent가 과거 기준으로 구현하지 않게 했습니다

5. 회사 비교실험과 SENTINEL ..... p. 8
품질 목표의 효과와 부담을 비교해 적용 기준을 정했습니다

목차 항목을 클릭하면 해당 위치로 이동합니다.

세부 항목은 PDF 책갈피에서, 목차 복귀는 각 페이지 하단에서 이용할 수 있습니다.

AI Engineer
자기소개서

황 화 인

포트폴리오(PPT) | 포트폴리오(Notion) | 이력서 | 개발블로그 | 링크드인

생성형 AI는 할루시네이션을 가지고 있으며 이는 컨텍스트 누적시 더 심해집니다. 따라서 이 직무에서 가장 중요한 역량은 제 1원리에 의해 계속해서 의심하고 검증하는 공학적 사고라고 생각합니다. 저는 AI가 내놓은 답도, 제가 선택한 방식도 직접 확인해야 납득합니다. 그래서 대안을 비교하고, 확인한 결과에 따라 구현과 적용 범위를 바꿔 왔습니다. 부동산 분석 Agent에서는 기술 구성을 바꿔가며 효과를 비교했고, 게임 NPC에서는 잘못된 답변의 원인을 저장된 기억까지 추적했습니다. 베스핀글로벌에서는 이 방식을 제품의 실행 결과 검증과 코딩 Agent의 작업 통제로 확장했습니다. 선택한 방식이 실제 문제에 맞는지 묻고, 평가 지표와 실제 동작을 근거로 구현과 판단을 수정하며 개발해 왔습니다.

1. ALL_FOR_ONE:
좋은 기법이 내 문제에서도 좋은지 확인했습니다

외부기관 협업 프로젝트 ALL_FOR_ONE에서는 부동산 현업자가 신뢰할 분양성 검토 보고서를 만들고자 했습니다. 저는 부동산 마케팅 기획 경험을 바탕으로 한국부동산마케팅협회에 자문을 구하고 실무 보고서 7건을 분석했습니다. 보고서 작성에는 1~2주가 걸렸고, 기존 자동화 결과는 데이터를 나열해도 해당 사업지와 시점에 맞는 해석이 부족하다는 의견을 들었습니다. 저는 출처와 판단 근거를 함께 제공하도록 DB와 처리 구조를 설계하고, LangGraph로 역할별 Agent를 연결했습니다. 검색은 pgvector의 의미 검색과 PGroonga의 키워드 검색을 결합해 정책의 정확한 수치와 표현이 다른 유사 정책을 함께 찾도록 구성했습니다.

마감이 앞당겨졌을 때는 의뢰자에게 가장 필요한 결과를 다시 물었습니다. 정책 보고서가 우선이라는 답을 듣고, 정책 분석과 각 Agent의 결과를 엮는 최종 보고서에 개발 시간을 집중했습니다. 보고서를 네 구간으로 나눠 작성하던 중에는 네 번째 호출에서 입력 한도를 넘겨 생성이 중단됐습니다. 메시지 누적 과정을 확인하니 약 7만 토큰의 기초자료가 매 호출에 다시 들어가 28만 토큰까지 쌓이고 있었습니다. 저는 첫 호출에만 전체 자료를 넣고, 이후에는 기존 메시지를 유지하면서 이전 구간 요약과 작성 지시만 추가하도록 바꿨습니다. 입력은 약 8만 토큰으로 줄었고 보고서를 끝까지 생성했습니다.

논문에서 효과가 보고된 ReAct가 부동산 분석에도 도움이 되는지는 제 환경에서 확인해야 한다고 판단했습니다. 저는 추론과 도구 실행을 반복하는 ReAct 구조에 자기검증을 결합하고, 전체 적용·관련 구성 제거·역할별 선택 적용의 세 구성을 DeepEval 9개 지표로 비교했습니다. 관련 구성을 제거했을 때 단순 추출형 Agent 두 개는 평가 점수가 높아졌지만, 복합 분석형에서는 낮아지는 경우가 나타났습니다. 저는 이 결과를 바탕으로 Agent의 역할에 따라 적용 범위를 나눴습니다.

선택 적용 후에도 주변시장 Agent의 점수가 떨어져 출력을 확인했습니다. 모델이 자기검증 도구를 실제로 호출하지 않고, 호출할 내용을 평문으로 출력하면서 내부 점검 문장이 보고서에 섞이고 있었습니다. 저는 ReAct 루프 뒤에 출력 정리 단계를 추가해 해당 문장을 제거하고, Structured Output의 result 항목에서 보고서 본문만 추출했습니다. 역할별 선택 적용과 출력 경로 수정을 반영한 구성의 종합 평가 점수는 79.04%에서 83.52%로 높아졌습니다.

프로젝트는 현업 전문가의 검토를 거쳐 2025년 부동산산업의 날에서 시연했으며, 전시 내용은 중앙일보와 한국경제의 행사 보도에 소개됐습니다. 이 경험 이후에는 기법의 이름이나 논문 결과만으로 적용 범위를 결정하지 않습니다. 어떤 입력에서 효과가 있고, 어디에서 불필요한 처리가 생기는지 제 환경에서 직접 확인합니다.

2. MEMORIA LABYRINTH:

응답의 문제를 기억이 만들어지는 단계까지 내려가 해결했습니다

MEMORIA LABYRINTH는 언리얼 개발자들과 협업해 만든 게임 프로젝트입니다. 저는 플레이어와 나눈 대화를 기억하고 관계 변화에 따라 다르게 반응하는 NPC의 대화·기억 기능을 맡았습니다. 초기에는 대화의 핵심 사실을 추출하고 중복 기억을 정리하는 Mem0를 적용했습니다. 그런데 NPC가 사용자의 정보를 자기 정보처럼 말해 DB를 확인하니, 당시 구성에는 "딸기를 좋아함"처럼 사실만 남고 누가 좋아하는지는 빠져 있었습니다. 플레이어의 말과 NPC의 말을 모두 기억해야 하는데 저장 단계에서 주체가 사라져, 검색한 기억을 응답에 넣어도 혼동이 이어졌습니다.

저는 Mem0가 챗봇에 맞춰져 있어서 이 프로젝트와 맞지 않다고 판단했습니다 따라서 Mem0의 기억 처리를 PostgreSQL 기반 장기기억 시스템으로 직접 구현해 대체했습니다. 대화에서 핵심 사실을 주어·관계·대상인 SPO 구조로 추출하고, 말한 사람과 그 사실의 주체를 함께 저장했습니다. 대화 원문은 세션 복원을 위해 별도 테이블에 보존하고, 기억 검색에는 추출한 사실만 사용하도록 나눴습니다. 취향이 바뀌면 이전 사실에 무효화 시점인 invalid_at을 기록해 원본은 남기되 현재 검색에서는 제외했습니다. 그리고 "사과를 좋아해" 다음에 "사과를 싫어해"라고 말한 대화에서는 NPC가 최신 선호를 기준으로 답하는 것을 확인했습니다.

기억이 저장돼 있는데도 답하지 못하는 경우도 있었습니다. DB에는 포도를 좋아한다는 정보가 있었지만 "내가 좋아하는 과일이 뭐였지?"라고 물으면 회상하지 못했습니다. 저는 사실을 추출할 때 포도와 함께 과일·음식 같은 상위 범주 키워드를 저장하고, 이를 임베딩과 키워드 검색에 함께 반영했습니다. 매 질문마다 LLM을 추가 호출해 검색어를 확장하면 응답이 늦어질 수 있어 백그라운드 저장 단계에서 처리했습니다. 수정 후 사과 선호를 저장한 대화에서는 "좋아하는 과일"이라는 질문에 사과를 답하는 것을 확인했습니다.

맨 처음 언리얼 팀이 보낸 통신 규격에는 호감도만 있고 기억해금도가 없었습니다. 저는 이대로면 호감도가 떨어질 때 이미 되찾은 기억도 다시 잠기는 구조가 된다고 설명했습니다. 그리고 만일 호감도가 떨어지지 않게 하면 플레이어의 말에 따라 NPC의 감정이 달라지는 경험을 보여주기 어렵다고 설득했습니다. 이 프로젝트의 본질은 몰입감이라고 판단했기 때문입니다. 결국 대화에 따라 감정은 바뀌되 되찾은 기억은 유지한다는 기획에 맞춰, 팀과 두 값을 별도 파라미터로 나누기로 합의했습니다. 그리고 팀은 "AI 기술력과 기획의 융합이 훌륭하다"는 평가를 받으며, 이 프로젝트로 최우수상인 1위를 받았습니다.

3. 베스핀글로벌 Copilot 평가·성능 개선:

평가 기준과 개발 Agent의 실행 경로를 함께 관리했습니다

베스핀글로벌의 Helpnow Foundry Copilot은 사용자가 “결재완료 메일이 오면 내용을 시트에 적재해줘”라고 요청하면 Gmail 읽기·시트 쓰기 등의 블록을 골라 연결하고 설정을 채워주는 워크플로우 빌더 Agent입니다. 저는 이 기능의 평가·개선 작업을 맡았습니다. 여기서 Copilot은 평가 대상인 제품이고, 제품 코드를 수정하는 코딩 Agent는 별도로 사용했습니다.

직접 사용해보니 화면에는 워크플로우가 만들어졌지만 실제로 작동하지 않거나, 수정 요청이 제대로 반영되지 않는 경우가 있었습니다. 버그를 보일 때마다 하나씩 고치는 사후 수정 방식만으로는 수많은 예외를 잡을 수 없고, 이전보다 나아졌는지 비교하기도 어려웠습니다. 저는 오류 재발을 근본적으로 막고자 실제 고객 빈도가 높은 요청 패턴 100개 시나리오가 실수 없이 완벽히 동작하는 것을 목표로 삼았습니다. 각 시나리오의 입력 자료와 외부 API·DB 실제 결과를 끝까지 검증하는 체계를 설계하고, 수정 전후 성과를 수치로 대조할 수 있는 정량적 기준선(Baseline)을 확립했습니다.

예를 들어 포상 대상자에게 메일을 보내는 요청이라면 메일 블록의 존재만 검사해서는 안 됐습니다. 대상자에게 메일이 갔는지, 비대상자에게는 가지 않았는지, 발송 내역이 시트에 남았는지를 각각 확인했습니다. Gmail API로 결과를 다시 읽되 실행 시각과 수신자를 조건으로 삼아, 과거 메일을 이번 실행의 성공으로 세지 않도록 했습니다. 시트와 문서는 시험 자료의 고유번호로 조회했습니다.

평가기 자체에도 문제가 있었습니다. 자동 실행을 위한 배포를 시도하지 않은 22건이 통과로 집계되고 있었습니다. 실패 응답이 없다는 이유로 성공 처리한 것입니다. 미시도·성공·서버 응답 실패·실행 예외를 분리하고, 연결 검사도 제품이 보정하기 전의 계획이 아니라 DB에 실제 저장된 연결을 읽도록 바꿨습니다. 기능 합격은 필수 검사와 실제 결과 확인으로 결정하고, LLM Judge의 설명 품질 점수는 참고값으로 분리했습니다.

수정 작업은 여러 저장소와 여러 대화 세션에 걸쳐 진행됐습니다. 새 세션이 오래된 평가기를 실행하거나 서버를 다른 방법으로 띄우면, 같은 코드를 고치더라도 비교하는 점수와 찾아야 할 로그가 달라졌습니다. 저는 새 세션이 어디서부터 읽고 어떤 상태에서 작업을 이어갈지 정할 필요가 있었습니다.

`SessionStart`에서는 `AGENTS.md → spec.md → repair-loop.md` 순서로 읽도록 안내했습니다. `AGENTS.md`는 문서 위치와 기본 규칙을 찾는 색인, `spec.md`는 완료 조건과 수정 금지 범위, `repair-loop.md`는 실패를 고르고 제품·평가기 원인을 분류한 뒤 수정·재검사하는 절차를 맡겼습니다. 모든 내용을 한 문서에 계속 붙이면 지시 파일이 길어지고 현재 필요한 기준을 찾기 어려워져, 문서의 역할을 나누고 읽는 순서를 연결했습니다.

문서만 안내해서는 현재 어디까지 진행했는지 알 수 없었습니다. 그래서 최신 리포트의 통과 수와 주요 실패, 서비스별 실행 상태, `fixes/`에 남아 있는 미커밋 수정 기록 수 중 조회 가능한 정보를 함께 전달했습니다. 최근 실패는 다음 조사 지점을 고르는 데 필요했고, 서버 상태는 서버가 꺼져 생긴 실패와 제품 결함을 구분하는 데 필요했습니다. 미커밋 수정 기록은 이전 세션이 남겼지만 아직 변경 이력에 저장하지 않은 작업을 확인하기 위한 정보였습니다. 훅은 이 정보를 전달하고, 코딩 Agent가 안내받은 문서를 읽어 작업을 이어가도록 했습니다.

실행해서는 안 되는 명령은 `PreToolUse`에서 별도로 검사했습니다. 제가 고정해야 했던 것은 수정 전후를 비교할 시험지와 평가 경로였습니다. 제품을 고치는 과정에서 시험지의 입력이나 기대 결과까지 바꾸면 점수가 올라도 무엇이 개선됐는지 알 수 없었습니다. 옛 TypeScript 평가기를 실행하면 새로 추가한 외부 결과 검증이 빠진 점수로 판단하게 됐습니다.

따라서 대상 작업 공간의 Bash 명령에서 고정 시험지 수정, 옛 평가기 직접 호출, 서버 직접 기동·중지에 해당하는 패턴을 검사해 차단했습니다. 차단할 때는 이유와 함께 정식 평가기인 pytest 또는 서버 관리 도구 스크립트의 사용 경로를 돌려줬습니다. Agent가 그 안내에 맞춰 명령을 수정하고 다시 요청하면 재검사하도록 했습니다. 서버 관리 경로를 통일한 이유도 세션마다 실행 방식과 로그 위치가 달라지지 않게 하기 위해서였습니다.

커밋 직전에는 변경 대상에 맞는 검사를 추가했습니다. 제품 저장소의 평가용 브랜치에서 DB 구조를 변경하는 커밋에는 DB관련 Lint를 실행해, 마이그레이션 변경분에 기존 테이블·컬럼을 삭제하는 코드가 추가됐는지 검사했습니다. 정상적인 롤백에 필요한 되돌리기 함수의 삭제는 제외했습니다. 평가 저장소에서는 문서관련 Lint로 문서 목록과 실제 파일의 일치, 끊긴 링크, 갱신 기한, `AGENTS.md`의 100줄 제한을 검사했습니다. 다음 세션이 없는 문서를 찾거나 오래된 안내를 따라 작업하는 문제까지 변경 시점에 확인하려 했습니다.

수정한 원인과 코드 위치, 검증 명령과 결과는 `fixes/`에 남겼습니다. 새 평가 결과는 별도 실행 폴더에 기록하고 기존 결과는 덮어쓰지 않도록 했습니다. 이렇게 제품 코드의 수정, 평가기의 수정, 시험 결과를 구분해 남겨야 이후의 통과 수를 같은 기준에서 해석할 수 있었습니다.

4. Q&A 게시판:

요구사항이 바뀌어도 코딩 Agent가 과거 기준으로 구현하지 않게 했습니다

회사에서는 고객 문의가 Google Chat에 흩어져 이미 답한 질문에도 다시 대응해야 하는 문제가 있었습니다. 저는 고객이 문의를 등록하고 기존 답변을 참고할 수 있는 Q&A 기능을 맡았습니다. 멘토와 요구사항·정책을 협의하고, 기존 로그인과 공통 UI를 활용해 Next.js 화면 6개, FastAPI의 API 12개, 문의·답변·첨부 테이블 3종을 연결했습니다.

첨부파일을 구현할 때는 DB와 MinIO의 상태가 어긋나는 실패를 확인했습니다. 정상 파일을 먼저 업로드한 뒤 다음 파일이 5MB 제한을 넘으면 요청은 실패했지만, 먼저 저장한 파일은 MinIO에 남았습니다. DB 트랜잭션이 롤백되어도 객체 저장소의 업로드까지 취소되지는 않기 때문입니다. 전체 파일을 선검증한 뒤 업로드하도록 순서를 바꾸고, 업로드나 DB 커밋이 실패하면 해당 요청의 객체를 보상 삭제하도록 했습니다. 보상 삭제까지 실패한 키는 로그에 남겼습니다.

기능을 코딩 Agent와 구현하면서는 요구사항을 어떤 형태로 전달할지도 설계했습니다. 요구사항 엑셀과 API·DB·화면 설계서를 매 세션 다시 탐색하게 하면 같은 시트와 표를 반복해서 읽게 됩니다. 그렇다고 Markdown으로 한 번 옮겨놓고 그것만 읽히면, 원본이 바뀌었을 때 과거 설계를 구현할 수 있었습니다.

저는 요구사항과 설계 원본을 기준으로 두고, 기능 ID로 화면·API·DB 명세를 연결한 작업용 Markdown을 추출했습니다. `SessionStart`에서 진행 중인 Jira 티켓과 함께 원본 수정 시각·추출본 생성 시각을 확인하고, 원본이 더 새로우면 추출본을 갱신해야 한다는 안내를 전달하도록 했습니다.

원본과 추출본이 다르면 원본을 우선하고, 설계 문서끼리 충돌하는 내용은 결정 기록으로 분리했습니다. 빠르게 읽을 작업용 문서와, 변경을 판단할 원본의 역할을 나눈 것입니다.

작업 범위는 Jira 티켓에 코드 위치, 수정 허용·금지 범위, 재현 절차, 완료 조건으로 명시했습니다. `jira-next`라는 skill에는 선행 티켓 완료 확인, 실제 코드 경로 대조, 계획 수립과 사람 승인, 구현·검증 순서를 두었습니다. API가 준비되지 않았는데 화면부터 구현하거나, 한 티켓을 고치면서 관련 없는 영역까지 넓히지 않게 하려는 절차였습니다. 검증에 실패하면 진행을 멈추고 남은 문제를 보고하며, 범위 밖 문제는 별도 티켓으로 분리하도록 했습니다.

작업 기준과 작업 결과도 같은 곳에 섞지 않았습니다. 마일스톤 문서를 다시 등록하면 Jira 본문이 갱신되므로, 구현 이력까지 본문에 쓰면 재동기화 과정에서 덮어쓸 수 있었습니다. 본문에는 요구사항과 완료 조건을, 코멘트에는 구현·검증 결과를 남기도록 분리했습니다.

커밋·푸시·PR 규칙은 세션 초반에 한 번 전달하고 방치하지 않았습니다.
context가 누적되다 보면 session 초반에 들어왔던 규칙들은 희미해지기 마련입니다. 하지만 그렇다고 Agent.md 문서에 적어서 매턴마다 그 많은 context를 추가하는 것도 문제였습니다. 따라서 `PreToolUse`에서 해당 명령을 감지하면 규칙 원본을 그 시점에 읽고 필수 항목과 사전 점검 목록을 적재적소에 Agent에 다시 전달했습니다. 규칙을 훅 코드에 복사해두면 문서와 훅을 따로 고쳐야 하므로, 원본 문서를 런타임에 읽도록 했습니다. 이 훅은 실행 직전의 규칙 재확인을 담당했습니다.

`Stop` 훅은 응답 종료 시 작업 요약 마커를 확인했습니다. 요약이 없으면 한 번 종료를 막고 작성을 요청한 뒤, 작성된 요약을 시각·세션 ID와 함께 작업 폴더에 누적했습니다. 긴 대화에만 남아 있던 완료 작업과 TODO를 파일로 남겨, 이후 작업에서 다시 확인할 수 있게 했습니다.

검증 도구도 Agent가 직접 사용할 수 있도록 연결했습니다. Playwright로 화면을 조작하고 API 응답과 서비스별 파일 로그를 확인하게 했습니다. 서버는 관련 스크립트가 systemd에 기동을 요청한 뒤, 포트를 점유한 프로세스가 해당 서비스 소속인지와 실제 응답을 함께 확인하도록 했습니다. 포트가 열려 있다는 이유만으로 다른 프로세스를 정상 서버로 오인하지 않기 위해서였습니다. 로그를 터미널 화면에만 두지 않고 고정 파일에 남긴 이유도, 세션이 바뀌어도 같은 오류의 스택 트레이스를 다시 읽을 수 있게 하기 위해서였습니다.

이 과정에서 제가 관리한 것은 Agent에게 전달할 작업 지시뿐만이 아니었습니다. 어느 버전의 요구사항을 읽었는지, 어떤 범위를 구현하는지, 무엇을 실행해 확인했는지, 다음 작업자가 어디서 그 근거를 찾을지까지 연결했습니다.

5. 회사 비교실험과 SENTINEL:

품질 목표의 효과와 부담을 비교해 적용 기준을 정했습니다

ALL_FOR_ONE(멀티에이전트 분양성 평가 보고서 프로젝트)에서 기술 구성을 비교했던 경험은 회사의 코딩 자동화 도구 검토로 이어졌습니다. 실제로 해결했던 백엔드 결함 3건을 수정 전 커밋으로 되돌리고, 모델과 하네스의 11개 구성을 총 99회 비교했습니다. 각 시행의 작업 환경을 분리하고, 정답이 되는 커밋과 숨긴 채점 테스트(제대로 코드를 짰다면 통과해야 할 테스트)가 문제 해결의 힌트로 섞이지 않도록 접근을 제한했습니다.

기능 판정에서는 결함을 재현하는 테스트가 모두 통과하고 기존 테스트의 신규 실패·소실이 없어야 해결로 인정했습니다. Agent가 마지막에 반환한 DONE도 이 판정과 대조했습니다. 코드 품질은 복잡도와 테스트 커버리지를 함께 보는 CRAP, 코드에 의도적인 변형을 넣어 테스트가 잡아내는지 확인하는 Mutation Testing으로 따로 측정했습니다. 이때 숨긴 채점 테스트는 품질 측정에서 제외하고, Agent가 수정한 코드와 작성한 테스트를 대상으로 삼았습니다.

그런데 앞선 비교에서는 품질 지표와 기능 해결의 관계가 강하지 않았습니다. 그리고 모델과 하네스가 서로 달랐으므로, 이 결과만으로 품질 목표가 도움이 되는지 판단하기 어려웠습니다. 품질 기준을 높이는 과정이 오히려 기능 해결을 방해하는지, 어느 수준까지 요구하는 것이 적절한지도 확인하고 싶었습니다.

그래서 Prime Agent와 deepseek-v4-flash를 고정하고, 품질 목표의 유무와 수준을 달리한 6개 조건을 54회 비교했습니다. 모든 조건에 회귀 테스트 작성·실행을 하도록 설계했고, 품질 목표를 준 조건에는 검사 도구 사용법과 목표 미달 시 수정·재측정 절차를 추가했습니다. 숫자 목표만 바꾼 실험이 아니라, 그 목표를 확인하고 다시 수정하는 작업 절차까지 포함한 비교였습니다.

그 결과 품질 지표는 좋아졌습니다. 기능해결과 관련해선 CRAP은 약한 상관관계를 보였으며 mutation 의 경우 상관관계가 거의 없는걸로 나타났습니다. 하지만 이로인해 작업시간이 크게 늘어났기에 두 지표는 기능 해결률을 높이기 위한 목적으로만 사용하기보다, 코드의 복잡도를 낮추고 테스트 커버리지와 결함 검출력을 높여 이후의 유지보수시 부담을 낮추고, 지뢰처럼 깔아놓은 테스트로 회귀를 방지하는 목적으로 활용해야 한다고 판단했습니다.
그리고 달리한 6개의 조건 중 가장 품질 지표가 나아진 2개의 조건 중 보다 약 22.7% 더 단축된 조건을 기준으로 선택해 오픈소스 품질 게이트 SENTINEL을 만들었습니다.

SENTINEL을 만든 계기는 이렇습니다. 누구나 코딩 에이전트를 활용하여 몇초만에 수백줄의 코드를 작성하는 세상에서 이제 우리가 단순히 코드가 돌아가느냐 보다 더욱 중요시될 부분이 바로 코드 퀄리티라고 생각했습니다.
그리고 CRAP과 mutation test 는 Coding Agent가 작성하는 코드의 복잡도, Test 의 커버리지, Test의 품질을 정량적으로 평가하는 결정론적 도구이므로 이 일에 적합한 도구하고 판단했습니다.
또한 실험에서 사용한 이 품질 확인 절차를 다른 프로젝트에서도 반복하려면, 언어별 도구의 출력 형식을 통일해서 매번 새로 해석하지 않도록 하는 것이 필요했습니다. 그래서 Python · TypeScript · Java의 기존 검사 도구를 연결하고, 검사 대상과 범위, 측정값, 기준 미달 위치를 공통 JSON으로 반환하도록 구성했습니다.

점수만 반환하는 것도 충분하지 않았습니다. 일부 함수만 검사하고 프로젝트 전체가 통과했다고 해석하거나, 실행 오류로 측정하지 못했는데 문제가 없다고 받아들이면 다음 수정의 방향도 틀어지기 때문입니다. 그래서 부분 · 전체 검사 범위를 표시하고, 건너뜀 · 기준 미달 · 실행 오류를 구분했습니다. 코딩 Agent가 어느 함수의 어떤 근거를 보고 수정해야 하는지 확인한 뒤 같은 범위를 다시 검사하도록 연결했습니다.

이 경험들에서 저는 드러난 결과에 머무르지 않고, 데이터가 만들어지는 단계와 도구가 실행되는 경로, 성공을 판단하는 기준까지 내려가서 문제의 원인을 찾았습니다. 문제에 맞게 구현과 작업 절차를 바꾸고, 그 효과를 실제 결과로 검증했습니다. 앞으로도 끊임없이 의심하며 문제가 왜 생겼고 어떻게 해결됐는지 설명할 수 있을 때까지 검증하여 완성도를 높여 나가겠습니다.